

ECC Skills Catalog

287 Claude Code skills, organized by category

Source: github.com/affaan-m/ECC (skills/ directory)

Healthcare / HIPAA-specific skills excluded from this catalog.

Installed skills live in `.claude/skill-library/` and are routed on demand via the `ecc-router` skill.

Contents

Contents	2
Agent & AI Orchestration	3
Language & Framework Patterns	11
Frontend, Design & UI	18
DevOps, Infra & Security	20
Networking & Homelab	23
Data, ML & Scientific Research	25
Video & Media Production	27
Business Ops, Finance & Legal	28
Marketing, Content & Growth	31
Git, Repo & Documentation	33
Integrations & Misc Tools	35

Agent & AI Orchestration

73 skills

agent-architecture-audit

Full-stack diagnostic for agent and LLM applications. Audits the 12-layer agent stack for wrapper regression, memory pollution, tool discipline failures, hidden repair loops, and rendering corruption. Produces severity-ranked findings with code-first fixes. Essential for developers building agent applications, autonomous loops, or any LLM-powered feature. Use when an agent or LLM feature misbehaves and the failing layer is unknown, or before shipping an agent stack.

agent-eval

Head-to-head comparison of coding agents (Claude Code, Aider, Codex, etc.) on custom tasks with pass rate, cost, time, and consistency metrics. Use when choosing between coding agents, or when a change to an agent setup needs measured pass rate, cost, and time rather than an impression.

agent-harness-construction

Design and optimize AI agent action spaces, tool definitions, and observation formatting for higher completion rates. Use when defining or revising an agent's tool set, action space, or observation format.

agent-introspection-debugging

Structured self-debugging workflow for AI agent failures using capture, diagnosis, contained recovery, and introspection reports. Use when an agent run fails and you need a reproducible diagnosis instead of a retry.

agent-payment-x402

Add x402 payment execution to AI agents with per-task budgets, spending controls, and non-custodial wallets. Supports Base through agentwallet-sdk and X Layer through OKX Payments / OKX Agent Payments Protocol. Use when an agent must pay for something itself and needs per-task budgets, spending controls, and a non-custodial wallet.

agent-self-evaluation

Use after completing any non-trivial task. The agent self-rates its output on 5 axes — accuracy, completeness, clarity, actionability, conciseness — with concrete evidence per criterion. Produces a structured 1-5 scorecard with specific improvement suggestions.

agent-sort

Build an evidence-backed ECC install plan for a specific repo by sorting skills, commands, rules, hooks, and extras into DAILY vs LIBRARY buckets using parallel repo-aware review passes. Use when ECC should be trimmed to what a project actually needs instead of loading the full bundle.

agentic-engineering

Operate as an agentic engineer using eval-first execution, decomposition, and cost-aware model routing. Use when planning or executing engineering work that agents will carry out end to end.

agentic-os

Build persistent multi-agent operating systems on Claude Code. Covers kernel architecture, specialist agents, slash commands, file-based memory, scheduled automation, and state management without external databases. Use when building a persistent multi-agent system on Claude Code with its own memory, commands, and scheduling.

ai-first-engineering

Engineering operating model for teams where AI agents generate a large share of implementation output. Use when setting team process, review gates, or ownership rules for a codebase largely written by agents.

ai-regression-testing

Regression testing strategies for AI-assisted development. Sandbox-mode API testing without database dependencies, automated bug-check workflows, and patterns to catch AI blind spots where the same model writes and reviews code. Use when adding regression coverage to AI-assisted code, or when the same model both wrote and reviewed a change.

autonomous-agent-harness

Transform Claude Code into a fully autonomous agent system with persistent memory, scheduled operations, computer use, and task queuing. Replaces standalone agent frameworks (Hermes, AutoGPT) by leveraging Claude Code's native crons, dispatch, MCP tools, and memory. Use when the user wants continuous autonomous operation, scheduled tasks, or a self-directing agent loop.

autonomous-loops

Patterns and architectures for autonomous Claude Code loops — from simple sequential pipelines to RFC-driven multi-agent DAG systems. Retained for compatibility only: when new autonomous loop guidance is needed, use continuous-agent-loop instead.

benchmark

Use this skill to measure performance baselines, detect regressions before/after PRs, and compare stack alternatives.

benchmark-methodology

Use after competitive-platform-analysis has produced a tiered competitor set. Scores each competitor across nine weighted dimensions (positioning, voice, visual craft, offer packaging, evidence, enterprise-readiness, thought leadership, pricing, client's strategic tension) with explicit 1 to 5 rubrics and a tension-plot. Precedes competitive-report-structure.

benchmark-optimization-loop

Use when the user asks to make something faster, try many variants, run recursive optimization, benchmark latency/throughput/cost, or choose the best implementation by repeated measured tests.

blueprint

Turn a one-line objective into a step-by-step construction plan for multi-session, multi-agent engineering projects. Each step has a self-contained context brief so a fresh agent can execute it cold. Includes adversarial review gate, dependency graph, parallel step detection, anti-pattern catalog, and plan mutation protocol. TRIGGER when: user requests a plan, blueprint, or roadmap for a complex multi-PR task, or describes work that needs multiple sessions. DO NOT TRIGGER when: task is completable in a single PR or fewer than 3 tool calls, or user says "just do it".

ck

Persistent per-project memory for Claude Code. Auto-loads project context on session start, tracks sessions with git activity, and writes to native memory. Commands run deterministic Node.js scripts — behavior is consistent across model versions. Use when a project needs context to survive across Claude Code sessions instead of being re-explained each time.

claude-devfleet

Orchestrate multi-agent coding tasks via Claude DevFleet — plan projects, dispatch parallel agents in isolated worktrees, monitor progress, and read structured reports. Use when dispatching parallel coding agents across isolated worktrees and tracking their reports.

configure-ecc

Guide ECC installation, update, or reconfiguration from inside Claude Code, Codex, or Kimi while respecting each harness's real plugin, scope, and hook capabilities.

context-budget

Audits Claude Code context window consumption across agents, skills, MCP servers, and rules. Identifies bloat, redundant components, and produces prioritized token-savings recommendations. Use when the context window is filling up too fast and the agents, skills, MCP servers, or rules consuming it need to be identified.

continuous-agent-loop

Patterns for continuous autonomous agent loops with quality gates, evals, and recovery controls. Use when running an agent loop that must self-check, gate on evals, and recover from failures.

continuous-learning

[DEPRECATED - use continuous-learning-v2] Legacy v1 stop-hook skill extractor. v2 is a strict superset with instinct-based, project-scoped, hook-reliable learning. Do not invoke v1: when continuous learning, session learning, or pattern extraction is requested, route to continuous-learning-v2 instead.

continuous-learning-v2

Instinct-based learning system that observes sessions via hooks, creates atomic instincts with confidence scoring, and evolves them into skills/commands/agents. v2.1 adds project-scoped instincts to prevent cross-project contamination. Use when capturing lessons from a session, managing instincts, or promoting them into skills, commands, or agents.

cost-aware-llm-pipeline

Cost optimization patterns for LLM API usage — model routing by task complexity, budget tracking, retry logic, and prompt caching. Use when LLM spend needs to come down, or when routing tasks across model tiers and budgets.

cost-tracking

Track and report Claude Code token usage, spending, and budgets from the local ECC cost-tracker metrics log. Use when the user asks about costs, spending, usage, tokens, budgets, or cost breakdowns by model, session, or date.

council

Convene a four-voice council for ambiguous decisions, tradeoffs, and go/no-go calls. Use when multiple valid paths exist and you need structured disagreement before choosing.

council-multi-model

Add one optional external Codex critique after the existing council has produced a decision draft. Use when an ambiguous, high-consequence decision would benefit from a separate model invocation's attempt to break the synthesis. Requires explicit consent before sending the compact draft and disagreement to OpenAI, labels same-provider reviews honestly, and marks the review absent when the adapter is unavailable.

delivery-gate

Stop hook that blocks Claude from finishing until quality checks pass. Detects rationalization patterns (surface text heuristics), stale learning logs (filesystem mtime), and low disk space. Complements self-audit by mechanically enforcing learning capture habits. Use when Claude should be mechanically blocked from declaring work finished before quality checks and learning capture actually pass.

dev-team

Simulate a collaborative dev team session where multiple role-based personas (PM, Architect, Developer, QA) respond to the same problem together in one session. Use when designing a feature, reviewing a proposal, or onboarding a new initiative and you want multi-role perspective without switching agents manually.

dmux-workflows

Multi-agent orchestration using dmux (tmux pane manager for AI agents). Patterns for parallel agent workflows across Claude Code, Codex, OpenCode, and other harnesses. Use when running multiple agent sessions in parallel or coordinating multi-agent development workflows.

dynamic-workflow-mode

Design task-local harnesses, eval gates, and reusable skill extraction for Claude dynamic workflow mode and other adaptive agent harnesses. Use when building a task-local harness, adding eval gates, or extracting a reusable skill from ad-hoc work.

ecc-guide

Guide users through ECC's current agents, skills, commands, hooks, rules, install profiles, and project onboarding by reading the live repository surface before answering.

ecc-recipes

Map a described workflow to the right ECC command-GROUP with run-order and stop condition, and browse all command-group recipe families. Adds a family-grouping + run-order + when-to-stop layer on top of the flat command catalog. Advisory only. TRIGGER when the user says which commands for X, what command group runs X, show ECC recipes, list ECC pipelines, or how do I run a workflow with ECC. DO NOT TRIGGER when the user wants the task executed directly, wants a single-command deep doc (use ecc-guide), or wants a draft prompt rewritten (use prompt-optimizer).

ecc-tools-cost-audit

Evidence-first ECC Tools burn and billing audit workflow. Use when investigating runaway PR creation, quota bypass, premium-model leakage, duplicate jobs, or GitHub App cost spikes in the ECC Tools repo.

enterprise-agent-ops

Operate long-lived agent workloads with observability, security boundaries, and lifecycle management. Use when running long-lived agent workloads that need observability, security boundaries, or lifecycle control.

eval-harness

Formal evaluation framework for Claude Code sessions implementing eval-driven development (EDD) principles. Use when a Claude Code workflow needs a formal eval before it is trusted or changed.

gan-style-harness

GAN-inspired Generator-Evaluator agent harness for building high-quality applications autonomously. Based on Anthropic's March 2026 harness design paper. Use when a feature should be built autonomously through generator and evaluator iteration until it clears a quality bar.

gateguard

it leaves.

orch-add-feature

Orchestrate building a brand-new feature end to end — research, plan, TDD implementation, review, and gated commit — by delegating each phase to the matching ECC agent. Use when adding a capability that does not exist yet.

orch-build-mvp

Orchestrate bootstrapping a working MVP from a design or spec document — ingest the doc, plan thin vertical slices, scaffold the first end-to-end slice, then TDD-implement, review, and gated commit. Use to turn an SDD/PRD into a running starting point. Use when a design or spec document must become a running MVP through planned vertical slices.

orch-change-feature

Orchestrate altering an existing, working feature to new desired behavior — update its tests to the new spec, change the implementation to match, review, and gated commit. Use when behavior is not broken but should be different.

orch-fix-defect

Orchestrate fixing a bug — reproduce it as a failing regression test, fix to green, review, and gated commit — by delegating each phase to the matching ECC agent. Use when existing behavior is broken or wrong.

orch-pipeline

Shared orchestration engine for the orch-* skill family. Defines the gated Research-Plan-TDD-Review-Commit pipeline, the size classifier, the agent map, and the two human gates that the orch-* operation skills delegate to. Not usually invoked directly. Not usually invoked directly; it applies when an orch-* skill delegates its gated Research-Plan-TDD-Review-Commit pipeline.

orch-refine-code

Orchestrate a behavior-preserving refactor — confirm tests are green, restructure without changing behavior, keep tests green, review, and gated commit. Use when the structure should improve but behavior must not change.

parallel-execution-optimizer

Use when the user wants a task done much faster through parallel work, concurrent agents, batched tool calls, isolated worktrees, or many independent verification lanes without losing correctness.

plan-canvas

Open plans and HTML artifacts in a local browser canvas where the human annotates elements, chats, and approves or requests changes without leaving the page. Use when presenting a plan for review, or when feedback like "move this, change that" is easier pointed at than typed.

plan-orchestrate

Read a plan document, decompose it into steps, design a per-step agent chain from the ECC catalogue, and emit ready-to-paste /orchestrate custom prompts. Generative only — never invokes /orchestrate itself. Use when the user has a multi-step plan and wants to drive it through orchestrate without composing chains by hand.

prompt-optimizer

Analyze raw prompts, identify intent and gaps, match ECC components (skills/commands/agents/hooks), and output a ready-to-paste optimized prompt. Advisory role only — never executes the task itself. TRIGGER when: user says "optimize prompt", "improve my prompt", "how to write a prompt for", "help me prompt", "rewrite this

prompt", or explicitly asks to enhance prompt quality. Also triggers on Chinese equivalents: "■■■prompt", "■■■prompt", "■■■prompt", "■■■■■■■■■■". DO NOT TRIGGER when: user wants the task executed directly, or says "just do it" / "■■■■". DO NOT TRIGGER when user says "■■■■■", "■■■■■", "optimize performance", "optimize this code" — those are refactoring/performance tasks, not prompt optimization.

ralphinho-rfc-pipeline

RFC-driven multi-agent DAG execution pattern with quality gates, merge queues, and work unit orchestration. Use when running RFC-driven multi-agent execution with quality gates and a merge queue.

recursive-decision-ledger

Use when the user asks for repeated rollouts, marked decision processes, high-dimensional search, stochastic optimization, local-optima exploration, ensemble comparison, or recursive reasoning with a visible evidence trail.

regex-vs-llm-structured-text

Decision framework for choosing between regex and LLM when parsing structured text — start with regex, add LLM only for low-confidence edge cases.

rules-distill

Scan skills to extract cross-cutting principles and distill them into rules — append, revise, or create new rule files. Use when the same principle keeps recurring across skills and belongs in a rule file instead.

safety-guard

Use this skill to prevent destructive operations when working on production systems or running agents autonomously.

santa-method

Multi-agent adversarial verification with convergence loop. Two independent review agents must both pass before output ships. Use when output must clear two independent adversarial reviewers before it ships.

search-first

Research-before-coding workflow. Search for existing tools, libraries, and patterns before writing custom code. Invokes the researcher agent.

skill-comply

Visualize whether skills, rules, and agent definitions are actually followed — auto-generates scenarios at 3 prompt strictness levels, runs agents, classifies behavioral sequences, and reports compliance rates with full tool call timelines. Use when checking whether agents actually follow the skills, rules, and definitions they were given, rather than assuming they do.

skill-scout

Search existing local, marketplace, GitHub, and web skill sources before creating a new skill. Use when the user wants to create, build, fork, or find a skill for a workflow.

skill-stocktake

Use when auditing Claude skills and commands for quality. Supports Quick Scan (changed skills only) and Full Stocktake modes with sequential subagent batch evaluation.

strategic-compact

Suggests manual context compaction at logical intervals to preserve context through task phases rather than arbitrary auto-compaction. Use when a session is approaching a context limit and a task phase is a natural place

to compact.

tdd-workflow

Use this skill when writing new features, fixing bugs, or refactoring code. Enforces test-driven development with 80%+ coverage including unit, integration, and E2E tests.

team-agent-orchestration

Run team-based orchestration for agent squads using work items, ownership, agent Kanban, merge gates, and control pane handoffs. Use when coordinating an agent squad with work items, ownership, Kanban, and merge gates.

team-builder

Interactive agent picker for composing and dispatching parallel teams. Use when composing and dispatching a parallel team of agents for a task.

token-budget-advisor

Offers the user an informed choice about how much response depth to consume before answering. Use this skill when the user explicitly wants to control response length, depth, or token budget. TRIGGER when: "token budget", "token count", "token usage", "token limit", "response length", "answer depth", "short version", "brief answer", "detailed answer", "exhaustive answer", "respuesta corta vs larga", "cuántos tokens", "ahorrar tokens", "responde al 50%", "dame la versión corta", "quiero controlar cuánto usas", or clear variants where the user is explicitly asking to control answer size or depth. DO NOT TRIGGER when: user has already specified a level in the current session (maintain it), the request is clearly a one-word answer, or "token" refers to auth/session/payment tokens rather than response size.

unified-memory

Share durable, inspectable context and handoffs between Claude, Codex, Hermes, Cursor, OpenCode, and other agents through the local ECC Memory Vault. Use when an agent must save work state, transfer context, resume another agent's task, or search shared project knowledge.

verification-loop

A comprehensive verification system for Claude Code sessions. Use when verifying a Claude Code session's work before claiming it is complete.

Language & Framework Patterns

73 skills

android-clean-architecture

Clean Architecture patterns for Android and Kotlin Multiplatform projects — module structure, dependency rules, UseCases, Repositories, and data layer patterns. Use when structuring modules, layers, or data flow in an Android or KMP project.

angular-developer

Generates Angular code and provides architectural guidance. Trigger when creating projects, components, or services, or for best practices on reactivity (signals, linkedSignal, resource), forms, dependency injection, routing, SSR, accessibility (ARIA), animations, styling (component styles, Tailwind CSS), testing, or CLI tooling.

backend-patterns

Backend architecture patterns, API design, database optimization, and server-side best practices for Node.js, Express, and Next.js API routes. Use when building or reviewing Node.js, Express, or Next.js API routes and their data access.

bun-runtime

Bun as runtime, package manager, bundler, and test runner. When to choose Bun vs Node, migration notes, and Vercel support.

coding-standards

Baseline cross-project coding conventions for naming, readability, immutability, and code-quality review. Use detailed frontend or backend skills for framework-specific patterns. Use when reviewing code quality or naming with no framework-specific skill that applies.

compose-multiplatform-patterns

Compose Multiplatform and Jetpack Compose patterns for KMP projects — state management, navigation, theming, performance, and platform-specific UI. Use when building Compose or Jetpack Compose UI, state, navigation, or theming in a KMP project.

cpp-coding-standards

C++ coding standards based on the C++ Core Guidelines (isocpp.github.io). Use when writing, reviewing, or refactoring C++ code to enforce modern, safe, and idiomatic practices.

cpp-testing

Use only when writing/updating/fixing C++ tests, configuring GoogleTest/CTest, diagnosing failing or flaky tests, or adding coverage/sanitizers.

csharp-testing

C# and .NET testing patterns with xUnit, FluentAssertions, mocking, integration tests, and test organization best practices. Use when writing or reviewing xUnit tests, mocks, or integration tests in a C# / .NET project.

dart-flutter-patterns

Production-ready Dart and Flutter patterns covering null safety, immutable state, async composition, widget architecture, popular state management frameworks (BLoC, Riverpod, Provider), GoRouter navigation, Dio networking, Freezed code generation, and clean architecture. Use when writing or reviewing Dart and Flutter code — state, widgets, navigation, networking, or architecture.

django-celery

Django + Celery async task patterns — configuration, task design, beat scheduling, retries, canvas workflows, monitoring, and testing. Use when adding background jobs, scheduled tasks, or async processing to a Django app.

django-patterns

Django architecture patterns, REST API design with DRF, ORM best practices, caching, signals, middleware, and production-grade Django apps. Use when building or reviewing Django apps, DRF APIs, ORM queries, or caching.

django-security

Django security best practices, authentication, authorization, CSRF protection, SQL injection prevention, XSS prevention, and secure deployment configurations. Use when reviewing Django authentication, authorization, input handling, or deployment settings.

django-tdd

Django testing strategies with pytest-django, TDD methodology, factory_boy, mocking, coverage, and testing Django REST Framework APIs. Use when writing Django or DRF tests with pytest-django, or driving a Django feature test-first.

django-verification

Verification loop for Django projects: migrations, linting, tests with coverage, security scans, and deployment readiness checks before release or PR.

dotnet-patterns

Idiomatic C# and .NET patterns, conventions, dependency injection, async/await, and best practices for building robust, maintainable .NET applications. Use when writing or reviewing C# / .NET code — DI, async, or general conventions.

error-handling

Patterns for robust error handling across TypeScript, Python, and Go. Covers typed errors, error boundaries, retries, circuit breakers, and user-facing error messages. Use when designing error types, retries, circuit breakers, or user-facing failure messages in TypeScript, Python, or Go.

evm-token-decimals

Prevent silent decimal mismatch bugs across EVM chains. Covers runtime decimal lookup, chain-aware caching, bridged-token precision drift, and safe normalization for bots, dashboards, and DeFi tools. Use when handling token amounts across EVM chains, or when a balance, price, or transfer amount is off by orders of magnitude.

fastapi-patterns

FastAPI best practices covering project structure, Pydantic v2 schemas, dependency injection, async handlers, authentication, authorization, transactional service layers, and testing with httpx and pytest. Use when building or reviewing FastAPI apps — Pydantic schemas, dependencies, async handlers, auth, or tests.

flutter-dart-code-review

Library-agnostic Flutter/Dart code review checklist covering widget best practices, state management patterns (BLoC, Riverpod, Provider, GetX, MobX, Signals), Dart idioms, performance, accessibility, security, and clean architecture. Use when reviewing Flutter or Dart code, whatever state management library the project uses.

foundation-models-on-device

Apple FoundationModels framework for on-device LLM — text generation, guided generation with @Generable, tool calling, and snapshot streaming in iOS 26+. Use when adding on-device LLM features with Apple

FoundationModels on iOS 26+.

fsharp-testing

F# testing patterns with xUnit, FsUnit, Unquote, FsCheck property-based testing, integration tests, and test organization best practices. Use when writing F# tests with xUnit, FsUnit, Unquote, or FsCheck.

golang-patterns

Idiomatic Go patterns, best practices, and conventions for building robust, efficient, and maintainable Go applications. Use when writing or reviewing Go code and idiomatic structure or conventions are in question.

golang-testing

Go testing patterns including table-driven tests, subtests, benchmarks, fuzzing, and test coverage. Follows TDD methodology with idiomatic Go practices. Use when writing Go tests — table-driven cases, subtests, benchmarks, fuzzing, or coverage.

hermes-imports

Convert local Hermes operator workflows into sanitized ECC skills and release-pack artifacts. Use when preparing a Hermes workflow for public ECC reuse without leaking private workspace state, credentials, or local-only paths.

hexagonal-architecture

Design, implement, and refactor Ports & Adapters systems with clear domain boundaries, dependency inversion, and testable use-case orchestration across TypeScript, Java, Kotlin, and Go services. Use when introducing or refactoring toward Ports and Adapters, or when domain logic has become entangled with I/O.

java-coding-standards

Java coding standards for Spring Boot and Quarkus services: naming, immutability, Optional usage, streams, exceptions, generics, CDI, reactive patterns, and project layout. Automatically applies framework-specific conventions. Use when writing or reviewing Java in a Spring Boot or Quarkus service.

jpa-patterns

JPA/Hibernate patterns for entity design, relationships, query optimization, transactions, auditing, indexing, pagination, and pooling in Spring Boot. Use when designing JPA entities or relationships, or when a Hibernate query, transaction, or N+1 problem needs fixing.

kotlin-coroutines-flows

Kotlin Coroutines and Flow patterns for Android and KMP — structured concurrency, Flow operators, StateFlow, error handling, and testing. Use when writing coroutines or Flow code on Android or KMP, or debugging cancellation and concurrency.

kotlin-exposed-patterns

JetBrains Exposed ORM patterns including DSL queries, DAO pattern, transactions, HikariCP connection pooling, Flyway migrations, and repository pattern. Use when working with the Exposed ORM — DSL or DAO queries, transactions, pooling, or migrations.

kotlin-ktor-patterns

Ktor server patterns including routing DSL, plugins, authentication, Koin DI, kotlinx.serialization, WebSockets, and testApplication testing. Use when building a Ktor server — routing, plugins, auth, DI, serialization, or tests.

kotlin-patterns

Idiomatic Kotlin patterns, best practices, and conventions for building robust, efficient, and maintainable Kotlin applications with coroutines, null safety, and DSL builders. Use when writing or reviewing Kotlin code and idiomatic structure or null safety is in question.

kotlin-testing

Kotlin testing patterns with Kotest, MockK, coroutine testing, property-based testing, and Kover coverage. Follows TDD methodology with idiomatic Kotlin practices. Use when writing Kotlin tests with Kotest or MockK, or testing coroutines and checking coverage.

laravel-patterns

Laravel architecture patterns, routing/controllers, Eloquent ORM, service layers, queues, events, caching, and API resources for production apps. Use when building or reviewing Laravel apps — controllers, Eloquent, service layers, queues, or API resources.

laravel-plugin-discovery

Discover and evaluate Laravel packages via LaraPlugins.io MCP. Use when the user wants to find plugins, check package health, or assess Laravel/PHP compatibility.

laravel-security

Laravel security best practices — authentication, authorization, Eloquent safety, CSRF, XSS prevention, API security, and secure deployment configurations. Use when reviewing Laravel auth, Eloquent safety, CSRF, XSS, API security, or deployment configuration.

laravel-tdd

Laravel testing strategies with PHPUnit, Pest, model factories, HTTP tests, Sanctum authentication testing, mocking, and coverage. Use when writing Laravel tests with PHPUnit or Pest, or driving a Laravel feature test-first.

laravel-verification

Verification loop for Laravel projects: env checks, linting, static analysis, tests with coverage, security scans, and deployment readiness. Use when verifying a Laravel project before merge or deploy — lint, static analysis, tests, coverage, security.

mysql-patterns

MySQL and MariaDB schema, query, indexing, transaction, replication, and connection-pool patterns for production backends. Use when designing MySQL or MariaDB schemas and indexes, or when a query, transaction, or replica lags.

nestjs-patterns

NestJS architecture patterns for modules, controllers, providers, DTO validation, guards, interceptors, config, and production-grade TypeScript backends. Use when building or reviewing a NestJS backend — modules, providers, DTO validation, guards, or interceptors.

nextjs-turbopack

Next.js 16+ and Turbopack — incremental bundling, FS caching, dev speed, and when to use Turbopack vs webpack.

nodejs-keccak256

Prevent Ethereum hashing bugs in JavaScript and TypeScript. Node's sha3-256 is NIST SHA3, not Ethereum Keccak-256, and silently breaks selectors, signatures, storage slots, and address derivation. Use when hashing for Ethereum in JavaScript or TypeScript, or when a selector, signature, storage slot, or derived address is wrong.

nuxt4-patterns

Nuxt 4 app patterns for hydration safety, performance, route rules, lazy loading, and SSR-safe data fetching with `useFetch` and `useAsyncData`. Use when building or reviewing a Nuxt 4 app, or debugging hydration mismatches and SSR-safe data fetching.

perl-patterns

Modern Perl 5.36+ idioms, best practices, and conventions for building robust, maintainable Perl applications. Use when writing or reviewing modern Perl 5.36+ code.

perl-security

Comprehensive Perl security covering taint mode, input validation, safe process execution, DBI parameterized queries, web security (XSS/SQLi/CSRF), and `perlcritic` security policies. Use when reviewing Perl input handling, process execution, DBI queries, or web-facing code.

perl-testing

Perl testing patterns using `Test2::V0`, `Test::More`, prove runner, mocking, coverage with `Devel::Cover`, and TDD methodology. Use when writing Perl tests with `Test2::V0` or `Test::More`, or measuring coverage.

postgres-patterns

PostgreSQL database patterns for query optimization, schema design, indexing, and security. Based on Supabase best practices. Use when designing PostgreSQL schemas, indexes, or RLS policies, or when a query is too slow.

prisma-patterns

Prisma ORM patterns for TypeScript backends — schema design, query optimization, transactions, pagination, and critical traps like `updateMany` returning count not records, `$transaction` timeouts, migrate dev resetting the DB, `@updatedAt` skipped on bulk writes, and serverless connection exhaustion. Use when writing a Prisma schema or query, or debugging transactions, migrations, or serverless connection limits.

python-patterns

Pythonic idioms, PEP 8 standards, type hints, and best practices for building robust, efficient, and maintainable Python applications. Use when writing or reviewing Python code and idiomatic structure, typing, or PEP 8 is in question.

python-testing

Python testing strategies using `pytest`, TDD methodology, fixtures, mocking, parametrization, and coverage requirements. Use when writing `pytest` tests — fixtures, mocks, parametrization, or coverage.

quarkus-patterns

Quarkus 3.x LTS architecture patterns with Camel for messaging, RESTful API design, CDI services, data access with Panache, and async processing. Use for Java Quarkus backend work with event-driven architectures. Use when building or reviewing a Quarkus service, especially with Camel messaging or Panache data access.

quarkus-security

Quarkus Security best practices for authentication, authorization, JWT/OIDC, RBAC, input validation, CSRF, secrets management, and dependency security. Use when reviewing Quarkus `authn/authz`, JWT or OIDC, RBAC, validation, or secrets.

quarkus-tdd

Test-driven development for Quarkus 3.x LTS using JUnit 5, Mockito, REST Assured, Camel testing, and JaCoCo. Use when adding features, fixing bugs, or refactoring event-driven services.

quarkus-verification

Verification loop for Quarkus projects: build, static analysis, tests with coverage, security scans, native compilation, and diff review before release or PR.

rails-patterns

Ruby on Rails framework patterns for Rails 7.1+ and 8.x apps. Covers the directory contract, skinny controllers with service objects, form objects, query objects, idiomatic ActiveRecord, background jobs, ViewComponent, Hotwire, and the Rails 8 Solid stack. Use when building or reviewing Rails apps, controllers, models, services, jobs, or views.

react-native-patterns

React Native and Expo app patterns — Expo Router navigation, state separation (server/client/route/form), TanStack Query data fetching with Zod, performant lists, NativeWind/StyleSheet styling, native APIs, and secure storage. Use when building or editing React Native / Expo screens, components, navigation, or data layers.

react-patterns

React 18/19 patterns including hooks discipline, server/client component boundaries, Suspense + error boundaries, form actions, data fetching, state management decision trees, and accessibility-first composition. Use when writing or reviewing React components.

react-performance

React and Next.js performance optimization patterns adapted from Vercel Engineering's React Best Practices (<https://github.com/vercel-labs/agent-skills>). Organizes 70+ rules across 8 priority categories — waterfalls, bundle size, server-side, client fetching, re-render, rendering, JS micro-perf, advanced. Use when writing, reviewing, or refactoring React/Next.js code for performance.

react-testing

React component testing with React Testing Library, Vitest/Jest, MSW for network mocking, accessibility assertions with axe, and the decision boundary between component tests and Playwright/Cypress end-to-end runs. Use when writing or fixing tests for React components, hooks, or pages.

redis-patterns

Redis data structure patterns, caching strategies, distributed locks, rate limiting, pub/sub, and connection management for production applications. Use when adding caching, a distributed lock, rate limiting, or pub/sub with Redis, or when key design needs review.

rust-patterns

Idiomatic Rust patterns, ownership, error handling, traits, concurrency, and best practices for building safe, performant applications. Use when writing or reviewing Rust code and ownership, error handling, traits, or concurrency is in question.

rust-testing

Rust testing patterns including unit tests, integration tests, async testing, property-based testing, mocking, and coverage. Follows TDD methodology. Use when writing Rust tests — unit, integration, async, property-based, or coverage.

springboot-patterns

Spring Boot architecture patterns, REST API design, layered services, data access, caching, async processing, and logging. Use for Java Spring Boot backend work. Use when building or reviewing a Spring Boot backend — REST layer, services, data access, caching, or async work.

springboot-security

Spring Security best practices for authn/authz, validation, CSRF, secrets, headers, rate limiting, and dependency security in Java Spring Boot services. Use when reviewing Spring Security authn/authz, validation, CSRF, secrets, headers, or rate limiting.

springboot-tdd

Test-driven development for Spring Boot using JUnit 5, Mockito, MockMvc, Testcontainers, and JaCoCo. Use when adding features, fixing bugs, or refactoring.

springboot-verification

Verification loop for Spring Boot projects: build, static analysis, tests with coverage, security scans, and diff review before release or PR.

swift-actor-persistence

Thread-safe data persistence in Swift using actors — in-memory cache with file-backed storage, eliminating data races by design. Use when persisting data in Swift and a data race or thread-safety problem needs designing out.

swift-concurrency-6-2

Swift 6.2 Approachable Concurrency — single-threaded by default, `@concurrent` for explicit background offloading, isolated conformances for main actor types. Use when adopting Swift 6.2 concurrency — offloading with `@concurrent` or resolving main-actor isolation.

swift-protocol-di-testing

Protocol-based dependency injection for testable Swift code — mock file system, network, and external APIs using focused protocols and Swift Testing. Use when Swift code needs testing and file system, network, or external APIs must be mocked.

swiftui-patterns

SwiftUI architecture patterns, state management with `@Observable`, view composition, navigation, performance optimization, and modern iOS/macOS UI best practices. Use when building or reviewing SwiftUI views, `@Observable` state, navigation, or render performance.

tinystuct-patterns

Expert guidance for developing with the tinystuct Java framework. Use when working on the tinystuct codebase or any project built on tinystuct — including creating Application classes, `@Action`-mapped routes, unit tests, ActionRegistry, HTTP/CLI dual-mode handling, the built-in HTTP server, the event system, JSON with Builder/Builders, database persistence with AbstractData, POJO generation, Server-Sent Events (SSE), file uploads, and outbound HTTP networking.

vite-patterns

Vite build tool patterns including config, plugins, HMR, env variables, proxy setup, SSR, library mode, dependency pre-bundling, and build optimization. Activate when working with vite.config.ts, Vite plugins, or Vite-based projects.

vue-patterns

Vue.js 3 Composition API patterns, component architecture, reactivity best practices, Pinia state management, Vue Router navigation, and Nuxt SSR patterns. Activates for Vue, Nuxt, Vite, or Pinia projects. Use when building or reviewing Vue 3, Nuxt, or Pinia code — Composition API, reactivity, or router navigation.

Frontend, Design & UI

17 skills

accessibility

Design, implement, and audit inclusive digital products using WCAG 2.2 Level AA. Use when building or auditing UI that must meet WCAG 2.2 Level AA, or when reviewing a change for keyboard, contrast, or screen-reader support. standards. Use this skill to generate semantic ARIA for Web and accessibility traits for Web and Native platforms (iOS/Android).

design-system

Use this skill to generate or audit design systems, check visual consistency, and review PRs that touch styling. Use when generating or auditing a design system, checking visual consistency, or reviewing a PR that touches styling.

frontend-a11y

Accessibility patterns for React and Next.js — semantic HTML, ARIA attributes, form labeling, keyboard navigation, focus management, and screen reader support. Use when building any interactive UI component or form.

frontend-design-direction

Set an ECC-specific frontend design direction for production UI work. Use when building or improving websites, dashboards, applications, components, landing pages, visual tools, or any web UI that needs stronger product-specific design judgment.

frontend-patterns

Frontend development patterns for React, Next.js, state management, performance optimization, and UI best practices. Use when building or reviewing React or Next.js components, state, or render performance.

frontend-slides

Create stunning, animation-rich HTML presentations from scratch or by converting PowerPoint files. Use when the user wants to build a presentation, convert a PPT/PPTX to web, or create slides for a talk/pitch. Helps non-designers discover their aesthetic through visual exploration rather than abstract choices.

ios-icon-gen

Generate iOS app icons as PNG imagesets for Xcode asset catalogs from SF Symbols (5000+ Apple-native) or Iconify API (275k+ open source icons from 200+ collections). Use when generating icons, creating icon assets, adding icons to asset catalog, or searching for icons for iOS projects.

liquid-glass-design

iOS 26 Liquid Glass design system — dynamic glass material with blur, reflection, and interactive morphing for SwiftUI, UIKit, and WidgetKit. Use when building iOS 26 Liquid Glass UI in SwiftUI, UIKit, or WidgetKit.

make-interfaces-feel-better

Apply concrete design-engineering details that make interfaces feel polished. Use when reviewing or improving UI spacing, typography, borders, shadows, motion, hit areas, icons, text wrapping, and interaction states.

motion-advanced

Advanced motion patterns for React / Next.js — drag & drop, gestures, text animations, SVG path drawing, custom hooks, imperative sequences (useAnimate), loaders, and the full API decision tree. Requires motion-foundations.

Use when building drag and drop, gestures, text or SVG animation, or imperative animation sequences in React or Next.js.

motion-foundations

Motion tokens, spring presets, performance rules, device adaptation, accessibility enforcement, and SSR safety for React / Next.js using motion/react. Foundation layer — all other motion skills depend on this. Use when setting up motion tokens, spring presets, reduced-motion handling, or SSR-safe animation in React or Next.js.

motion-patterns

Production-ready animation patterns for React / Next.js — button, modal, toast, stagger, page transitions, exit animations, scroll, and layout — built on motion-foundations tokens and springs. Use when animating a specific UI element in React or Next.js — button, modal, toast, stagger, page transition, or scroll.

taste

A creative-direction (taste) layer for music videos and short-form edits in the angelcore / cloud-trance / hyperpop visual family. Distills a named-genre aesthetic vocabulary, a mood + color + light system, and a beat-synced editing grammar, then chains ECC's video skills (video-editing, fal-ai-media, remotion-video-creation, motion-*, content-engine) into one production pipeline. Use when the work is not just making a video function but making it feel intentional, when building a music video, a fancam/edit, a moodboard-driven reel, or when choosing a coherent visual direction for AI-generated b-roll.

taste-application

Generate new video against a distilled style pack and cut it into a finished piece - plan takes from the reference's cut rhythm, generate on fal, grade with the pack's measured LUT, cut at the measured cadence, weave in existing footage, composite overlay plates, mint 3D props, and verify the result numerically. Use when the user wants to make a video in a captured style, supplement existing footage, or assemble generated clips into a real edit.

taste-distillation

Measure a set of reference videos into a reusable style pack - colour grade as a 3D LUT, cut rhythm as a shot-length distribution, hero stills, screen-blend overlay plates, and a text spec for a generative model. Use when the user wants to capture the look of reference footage, build a repeatable look, mint assets from references, or reproduce someone's grade and pacing.

ui-demo

Record polished UI demo videos using Playwright. Use when the user asks to create a demo, walkthrough, screen recording, or tutorial video of a web application. Produces WebM videos with visible cursor, natural pacing, and professional feel.

ui-to-vue

Use when the user has UI screenshots or design exports that need batch conversion into Vue 3 components, especially with Vant, Element Plus, or Ant Design Vue.

DevOps, Infra & Security

24 skills

automation-audit-ops

Evidence-first automation inventory and overlap audit workflow for ECC. Use when the user wants to know which jobs, hooks, connectors, MCP servers, or wrappers are live, broken, redundant, or missing before fixing anything.

browser-qa

Use this skill to automate visual testing and UI interaction verification using browser automation after deploying features.

canary-watch

Use this skill to monitor and verify a deployed URL after releases — checks HTTP endpoints, SSE streams, static assets, console errors, and performance regressions after deploys, merges, or dependency upgrades. Smoke / canary / post-deploy verification.

click-path-audit

Trace every user-facing button/touchpoint through its full state change sequence to find bugs where functions individually work but cancel each other out, produce wrong final state, or leave the UI in an inconsistent state. Use when: systematic debugging found no bugs but users report broken buttons, or after any major refactor touching shared state stores.

config-gc

Garbage collection for your Claude Code configuration. Periodically scans ~/.claude (skills, memory, hooks, permissions, MCP servers, caches) for redundant, stale, orphaned, or low-value items, then walks the user through a confirm-each-deletion cleanup. Use when the user says "clean up my config", "config GC", "too many skills", "audit my setup", "my .claude is bloated", or asks for a periodic config review.

content-hash-cache-pattern

Cache expensive file processing results using SHA-256 content hashes — path-independent, auto-invalidating, with service layer separation. Use when repeated file processing is slow and results should be cached and invalidated by content rather than path.

database-migrations

Database migration best practices for schema changes, data migrations, rollbacks, and zero-downtime deployments across PostgreSQL, MySQL, and common ORMs (Prisma, Drizzle, Kysely, Django, TypeORM, golang-migrate). Use when writing a schema or data migration, planning a rollback, or aiming for zero-downtime deployment.

defi-amm-security

Security checklist for Solidity AMM contracts, liquidity pools, and swap flows. Covers reentrancy, CEI ordering, donation or inflation attacks, oracle manipulation, slippage, admin controls, and integer math. Use when auditing or writing Solidity AMM, liquidity pool, or swap code.

deployment-patterns

Deployment workflows, CI/CD pipeline patterns, Docker containerization, health checks, rollback strategies, and production readiness checklists for web applications. Use when setting up CI/CD, containerizing an app, or checking production readiness before a release.

Use this skill when adding authentication, handling user input, working with secrets, creating API endpoints, or implementing payment/sensitive features. Provides comprehensive security checklist and patterns.

security-scan

Scan your Claude Code configuration (.claude/ directory) for security vulnerabilities, misconfigurations, and injection risks using AgentShield. Checks CLAUDE.md, settings.json, MCP servers, hooks, and agent definitions. Use when auditing a .claude/ directory — CLAUDE.md, settings.json, MCP servers, hooks, or agent definitions.

terminal-opener

Open an executable and its argument array in a visible terminal window through a reusable, shell-free launch plan with dry-run, JSON, capability detection, detached fallback, and standalone recovery modes. Use when Codex needs to open an interactive CLI, SSH session, local development process, sandbox, or other argv-based command in a new host terminal; diagnose whether a supported terminal is available; or provide an actionable plan when the requested terminal is unsupported.

terminal-ops

Evidence-first repo execution workflow for ECC. Use when the user wants a command run, a repo checked, a CI failure debugged, or a narrow fix pushed with exact proof of what was executed and verified.

uncloud

Use when managing an Uncloud cluster — deploying services, configuring Caddy ingress, adding static proxy routes for non-cluster devices, publishing ports, scaling, inspecting logs, or managing machines and volumes with the `uc` CLI.

windows-desktop-e2e

E2E testing for Windows native desktop apps (WPF, WinForms, Win32/MFC, Qt) using pywinauto and Windows UI Automation. Use when writing E2E tests for a Windows native desktop app with pywinauto or UI Automation.

Networking & Homelab

10 skills

cisco-ios-patterns

Cisco IOS and IOS-XE review patterns for show commands, config hierarchy, wildcard masks, ACL placement, interface hygiene, and safe change-window verification. Use when reading, writing, or reviewing Cisco IOS / IOS-XE configuration or planning a change window.

homelab-network-readiness

Readiness checklist for homelab VLAN segmentation, local DNS filtering, and WireGuard-style remote access before changing router, firewall, DHCP, or VPN configuration.

homelab-network-setup

Practical home and homelab network planning for gateways, switches, access points, IP ranges, DHCP reservations, DNS, cabling, and common beginner mistakes. Use when planning or fixing a home or homelab network — gateway, switch, AP, IP ranges, DHCP, DNS, or cabling.

homelab-pihole-dns

Pi-hole installation, blocklist management, DNS-over-HTTPS setup, DHCP integration, local DNS records, and troubleshooting broken DNS resolution on a home network. Use when the task explicitly involves Pi-hole — installing it, managing blocklists, configuring DoH or DHCP, adding local DNS records, or diagnosing DNS resolution with Pi-hole in the path.

homelab-vlan-segmentation

Segmenting home networks into VLANs for IoT, guest, trusted, and server traffic using UniFi, pfSense/OPNsense, and MikroTik — including switch trunk config, firewall rules, and wireless SSID mapping. Use when splitting a home network into IoT, guest, trusted, and server VLANs on UniFi, pfSense/OPNsense, or MikroTik.

homelab-wireguard-vpn

WireGuard VPN server setup, peer configuration, key generation, split tunneling vs full tunnel routing, and remote access to a home network from mobile and laptop clients. Use when setting up WireGuard for remote access to a home network, or deciding between split and full tunnel routing.

netmiko-ssh-automation

Safe Python Netmiko patterns for read-only collection, bounded batch SSH, TextFSM parsing, guarded config changes, timeouts, and network automation error handling. Use when automating network device access with Python Netmiko, whether collecting state or pushing guarded config changes.

network-bgp-diagnostics

Diagnostics-only BGP troubleshooting patterns for neighbor state, route exchange, prefix policy, AS path inspection, and safe evidence collection. Use when a BGP neighbor is down, routes are missing, or prefix policy and AS path need inspection.

network-config-validation

Pre-deployment checks for router and switch configuration, including dangerous commands, duplicate addresses, subnet overlaps, stale references, management-plane risk, and IOS-style security hygiene. Use when reviewing a router or switch configuration before deployment.

network-interface-health

Diagnose interface errors, drops, CRCs, duplex mismatches, flapping, speed negotiation issues, and counter trends on routers, switches, and Linux hosts. Use when an interface shows errors, drops, CRCs, flapping, or a duplex or speed mismatch.

Data, ML & Scientific Research

16 skills

clickhouse-io

ClickHouse database patterns, query optimization, analytics, and data engineering best practices for high-performance analytical workloads. Use when writing ClickHouse schemas or queries, or when an analytical query is too slow.

data-scraper-agent

Build a fully automated AI-powered data collection agent for any public source — job boards, prices, news, GitHub, sports, anything. Runs on a schedule, enriches data with a free LLM (Gemini Flash), stores results in Notion/Sheets/Supabase, and learns from user feedback. Runs 100% free on GitHub Actions. Use when the user wants to monitor, collect, or track any public data automatically.

data-throughput-accelerator

Use when large data ingestion, backfill, export, ETL, warehouse loading, manifest catch-up, or table synchronization needs to become much faster while preserving data correctness.

iterative-retrieval

Pattern for progressively refining context retrieval to solve the subagent context problem. Use when a subagent lacks the context it needs and retrieval must be refined across passes.

ito-baskets

Read-only Itô basket and prediction-market data skill. Index the live basket catalog, compare a basket against user-supplied research or a watchlist, build a source-grounded market brief, or draft a non-executable planning worksheet. Use when a user asks to browse or index Itô baskets, compare a basket against notes or a thesis, research prediction-market events/venues/liquidity, or plan a basket or market idea without trading. Never advises, orders, trades, reserves, or executes.

ito-compute

Query live GPU inventory, submit an authenticated Itô fixed-rate RFQ, inspect RFQ or procurement status, revoke device credentials, and run explicitly gated node qualification through the separately installed canonical CLI. Use when a user asks to find H100/H200 capacity, request a fixed compute rate, check Itô compute status, validate GPU nodes, revoke Itô access, or rent or purchase GPU compute and needs the supported boundary explained.

ito-inference

Inspect the availability of model serving on a completed Itô compute booking and, when the canonical backend becomes available, hand off an explicitly confirmed serving manifest. Use after ito-compute has booked GPU nodes and the user asks for an OpenAI-compatible endpoint, ito-serve, hosted Kimi, or self-hosted open-weights inference. ECC implements no serving stack of its own.

ito-training

Inspect the availability of ML training on a completed Itô compute booking and, when the canonical backend becomes available, hand off an explicitly confirmed training manifest. Use after ito-compute has booked GPU nodes and the user wants pre-training, fine-tuning, or RL on that metal. ECC implements no training stack of its own.

ml-adoption-playbook

End-to-end methodology for AI agents and software engineers to add machine learning algorithms to existing non-ML codebases. Covers problem framing, data readiness, architectural decoupling, and baseline model integration. Use when adding a machine learning capability to a codebase that has none, from problem framing through a baseline model.

pytorch-patterns

PyTorch deep learning patterns and best practices for building robust, efficient, and reproducible training pipelines, model architectures, and data loading. Use when writing or reviewing PyTorch training loops, model architectures, or data loading, or when a run will not reproduce.

recsys-pipeline-architect

Design composable recommendation, ranking, and feed pipelines using the six-stage Source→Hydrator→Filter→Scorer→Selector→SideEffect framework popularized by xAI's open-sourced For You algorithm. Use this skill whenever the user is building any system that picks "the top K items for a (user, context)" — social feeds, content CMSs, RAG rerankers, task prioritizers, notification triage, search reranking, ad ranking.

scientific-db-pubmed-database

Direct PubMed and NCBI E-utilities search workflows for biomedical literature, MeSH queries, PMID lookup, citation retrieval, and API-backed literature monitoring. Use when a task needs biomedical literature from PubMed rather than general web search.

scientific-db-uspto-database

USPTO patent and trademark data workflow for official record lookup, PatentSearch queries, TSDR checks, assignment data, and reproducible IP research logs. Use when a task needs official United States patent or trademark records from USPTO systems.

scientific-pkg-gget

gget CLI and Python workflow for quick genomic database queries, sequence lookup, BLAST-style searches, enrichment checks, and reproducible bioinformatics evidence logs. Use when a task needs quick bioinformatics lookup across genomic reference databases with the gget CLI or Python package.

scientific-thinking-literature-review

Systematic literature-review workflow for academic, biomedical, technical, and scientific topics, including search planning, source screening, synthesis, citation checks, and evidence logging. Use when the task is to find, screen, synthesize, and cite a body of academic or technical literature.

scientific-thinking-scholar-evaluation

Structured scholarly-work evaluation for papers, proposals, literature reviews, methods sections, evidence quality, citation support, and research-writing feedback. Use when evaluating academic or scientific work — papers, proposals, methods sections, or evidence quality — against a repeatable rubric.

Video & Media Production

7 skills

blender-motion-state-inspection

Use this skill when inspecting Blender characters, rigs, poses, animation retargeting, ground contact, facing direction, or model-vs-motion alignment where screenshots alone are not enough.

fal-ai-media

Unified media generation via fal.ai MCP — image, video, and audio. Covers text-to-image (Nano Banana), text/image-to-video (Seedance, Kling, Veo 3), text-to-speech (CSM-1B), and video-to-audio (ThinkSound). Use when the user wants to generate images, videos, or audio with AI.

manim-video

Build reusable Manim explainers for technical concepts, graphs, system diagrams, and product walkthroughs, then hand off to the wider ECC video stack if needed. Use when the user wants a clean animated explainer rather than a generic talking-head script.

remotion-video-creation

Best practices for Remotion - Video creation in React. 29 domain-specific rules covering 3D, animations, audio, captions, charts, transitions, and more. Use when building video in React with Remotion — animations, audio, captions, charts, or transitions.

tasteforge-video

Use for file-driven multimodal image, video, and 3D-asset discovery; taste interviews; distill or apply workflows; style-pack validation; editable EDL/FCPXML export; provenance audits; and offline planning that must fail closed before provider generation.

video-editing

AI-assisted video editing workflows for cutting, structuring, and augmenting real footage. Covers the full pipeline from raw capture through FFmpeg, Remotion, ElevenLabs, fal.ai, and final polish in Descript or CapCut. Use when the user wants to edit video, cut footage, create vlogs, or build video content.

videodb

See, Understand, Act on video and audio. See- ingest from local files, URLs, RTSP/live feeds, or live record desktop; return realtime context and playable stream links. Understand- extract frames, build visual/semantic/temporal indexes, and search moments with timestamps and auto-clips. Act- transcode and normalize (codec, fps, resolution, aspect ratio), perform timeline edits (subtitles, text/image overlays, branding, audio overlays, dubbing, translation), generate media assets (image, audio, video), and create real time alerts for events from live streams or desktop capture. Use when ingesting, indexing, searching, editing, transcoding, or alerting on video or audio content.

Business Ops, Finance & Legal

18 skills

carrier-relationship-management

Codified expertise for managing carrier portfolios, negotiating freight rates, tracking carrier performance, allocating freight, and maintaining strategic carrier relationships. Informed by transportation managers with 15+ years experience. Includes scorecarding frameworks, RFP processes, market intelligence, and compliance vetting. Use when managing carriers, negotiating rates, evaluating carrier performance, or building freight strategies.

counterparty-channel-discipline

Per-channel strict prompts, mention gating, silent observation, and a communication autonomy policy for agents that sit in shared channels with external counterparties. Use when an agent joins group chats, shared channels, or DMs where outsiders can read every message and you need it to speak only when addressed, never leak internal context, and route risky content to draft-only approval.

customer-billing-ops

Operate customer billing workflows such as subscriptions, refunds, churn triage, billing-portal recovery, and plan analysis using connected billing tools like Stripe. Use when the user needs to help a customer, inspect subscription state, or manage revenue-impacting billing operations.

customs-trade-compliance

Codified expertise for customs documentation, tariff classification, duty optimization, restricted party screening, and regulatory compliance across multiple jurisdictions. Informed by trade compliance specialists with 15+ years experience. Includes HS classification logic, Incoterms application, FTA utilization, and penalty mitigation. Use when handling customs clearance, tariff classification, trade compliance, import/export documentation, or duty optimization.

energy-procurement

Codified expertise for electricity and gas procurement, tariff optimization, demand charge management, renewable PPA evaluation, and multi-facility energy cost management. Informed by energy procurement managers with 15+ years experience at large commercial and industrial consumers. Includes market structure analysis, hedging strategies, load profiling, and sustainability reporting frameworks. Use when procuring energy, optimizing tariffs, managing demand charges, evaluating PPAs, or developing energy strategies.

esign-field-placement

Deterministic method for placing signature, date, and text fields in a web e-signature composer through a browser automation session, using a fixed signature page, numeric Location panel coordinates instead of drag, and a save-as-draft default. Use when automating envelope preparation for generated agreements and you need repeatable field positions, correct per-recipient ownership, and a hard gate before anything is sent or signed.

finance-billing-ops

Evidence-first revenue, pricing, refunds, team-billing, and billing-model truth workflow for ECC. Use when the user wants a sales snapshot, pricing comparison, duplicate-charge diagnosis, or code-backed billing reality instead of generic payments advice.

inventory-demand-planning

Codified expertise for demand forecasting, safety stock optimization, replenishment planning, and promotional lift estimation at multi-location retailers. Informed by demand planners with 15+ years experience managing hundreds of SKUs. Includes forecasting method selection, ABC/XYZ analysis, seasonal transition management, and vendor

negotiation frameworks. Use when forecasting demand, setting safety stock, planning replenishment, managing promotions, or optimizing inventory levels.

investor-materials

Create and update pitch decks, one-pagers, investor memos, accelerator applications, financial models, and fundraising materials. Use when the user needs investor-facing documents, projections, use-of-funds tables, milestone plans, or materials that must stay internally consistent across multiple fundraising assets.

investor-outreach

Draft cold emails, warm intro blurbs, follow-ups, update emails, and investor communications for fundraising. Use when the user wants outreach to angels, VCs, strategic investors, or accelerators and needs concise, personalized, investor-facing messaging.

logistics-exception-management

Codified expertise for handling freight exceptions, shipment delays, damages, losses, and carrier disputes. Informed by logistics professionals with 15+ years operational experience. Includes escalation protocols, carrier-specific behaviors, claims procedures, and judgment frameworks. Use when handling shipping exceptions, freight claims, delivery issues, or carrier disputes.

master-agreement-generator

Generate review drafts of counterparty master agreements from one template plus a JSON spec, with role-selected clauses and a Schedule A workflow limited to the executed agreement's notice authority. Use when you need reproducible drafting and separately reviewed execution preparation.

prediction-market-oracle-research

Research prediction markets as data sources or oracle signals for products, agents, dashboards, and corporate decision intelligence. Use for source-grounded analysis of market-implied probabilities, caveats, and integration patterns without investment advice. Use when evaluating prediction markets as a data source or oracle signal for a product, agent, or dashboard.

prediction-market-risk-review

Review prediction-market, basket, oracle, and trading-agent workflows for compliance, safety, data-quality, privacy, and execution risk. Use before any workflow handles venue auth, user portfolio data, API keys, or trade planning.

production-scheduling

Codified expertise for production scheduling, job sequencing, line balancing, changeover optimization, and bottleneck resolution in discrete and batch manufacturing. Informed by production schedulers with 15+ years experience. Includes TOC/drum-buffer-rope, SMED, OEE analysis, disruption response frameworks, and ERP/MES interaction patterns. Use when scheduling production, resolving bottlenecks, optimizing changeovers, responding to disruptions, or balancing manufacturing lines.

quality-nonconformance

Codified expertise for quality control, non-conformance investigation, root cause analysis, corrective action, and supplier quality management in regulated manufacturing. Informed by quality engineers with 15+ years experience across FDA, IATF 16949, and AS9100 environments. Includes NCR lifecycle management, CAPA systems, SPC interpretation, and audit methodology. Use when investigating non-conformances, performing root cause analysis, managing CAPAs, interpreting SPC data, or handling supplier quality issues.

returns-reverse-logistics

Codified expertise for returns authorization, receipt and inspection, disposition decisions, refund processing, fraud detection, and warranty claims management. Informed by returns operations managers with 15+ years experience. Includes grading frameworks, disposition economics, fraud pattern recognition, and vendor recovery processes. Use when handling product returns, reverse logistics, refund decisions, return fraud detection, or warranty claims.

visa-doc-translate

Translate visa application documents (images) to English and create a bilingual PDF with original and translation. Use when visa application document images must be translated to English as a bilingual PDF.

Marketing, Content & Growth

17 skills

article-writing

Write articles, guides, blog posts, tutorials, newsletter issues, and other long-form content in a distinctive voice derived from supplied examples or brand guidance. Use when the user wants polished written content longer than a paragraph, especially when voice consistency, structure, and credibility matter.

brand-discovery

Use when a brand needs to discover or articulate its identity through structured multi-session interviews. Covers purpose, positioning, audience, personality, voice, narrative, and founder-brand tension across 8 modules using laddering, 5 Whys, and projective techniques. Produces a resumable session with disk-persisted state and a master brandbook (90_SYNTHESIS.md).

brand-voice

Build a source-derived writing style profile from real posts, essays, launch notes, docs, or site copy, then reuse that profile across content, outreach, and social workflows. Use when the user wants voice consistency without generic AI writing tropes.

competitive-platform-analysis

Use when scoping a competitive landscape — identifying, categorising, and score-filtering a competitor set before any benchmarking begins. Decides who counts as a competitor, which tier they belong to, and which sources to mine. First step in the three-skill competitive pipeline; precedes benchmark-methodology.

competitive-report-structure

Use after benchmark-methodology has produced scored competitor profile cards. Assembles findings into a decision-grade report: landscape map, competitor profiles, benchmarking matrix, white-space analysis, strategic recommendations, and team alignment trigger questions. Final step in the three-skill competitive pipeline.

content-engine

Create platform-native content systems for X, LinkedIn, TikTok, YouTube, newsletters, and repurposed multi-platform campaigns. Use when the user wants social posts, threads, scripts, content calendars, or one source asset adapted cleanly across platforms.

crosspost

Multi-platform content distribution across X, LinkedIn, Threads, and Bluesky. Adapts content per platform using content-engine patterns. Never posts identical content cross-platform. Use when the user wants to distribute content across social platforms.

deep-research

Multi-source deep research using firecrawl and exa MCPs. Searches the web, synthesizes findings, and delivers cited reports with source attribution. Use when the user wants thorough research on any topic with evidence and citations.

growth-log

Use after a complex task, failure, or when reviewing what was learned. Teaches how to write growth logs that extract reusable patterns — not diary entries.

lead-intelligence

AI-native lead intelligence and outreach pipeline. Replaces Apollo, Clay, and ZoomInfo with agent-powered signal scoring, mutual ranking, warm path discovery, source-derived voice modeling, and channel-specific outreach across email, LinkedIn, and X. Use when the user wants to find, qualify, and reach high-value contacts.

market-research

Conduct market research, competitive analysis, investor due diligence, and industry intelligence with source attribution and decision-oriented summaries. Use when the user wants market sizing, competitor comparisons, fund research, technology scans, or research that informs business decisions.

marketing-campaign

End-to-end marketing campaign planning and execution. Covers audience research, positioning, campaign angle definition, landing page copy, email sequences, social posts, ad copy, short-form video scripts, and content calendars. Use as the orchestration layer for multi-channel product launches. Use when planning or executing a multi-channel product launch, or producing landing page, email, social, or ad copy.

research-ops

Evidence-first current-state research workflow for ECC. Use when the user wants fresh facts, comparisons, enrichment, or a recommendation built from current public evidence and any supplied local context.

seo

Audit, plan, and implement SEO improvements across technical SEO, on-page optimization, structured data, Core Web Vitals, and content strategy. Use when the user wants better search visibility, SEO remediation, schema markup, sitemap/robots work, or keyword mapping.

social-graph-ranker

Weighted social-graph ranking for warm intro discovery, bridge scoring, and network gap analysis across X and LinkedIn. Use when the user wants the reusable graph-ranking engine itself, not the broader outreach or network-maintenance workflow layered on top of it.

social-publisher

Agent-driven scheduling and publishing of social media posts across 13 platforms via SocialClaw. Use when the user wants to publish to X, LinkedIn, Instagram, Facebook Pages, TikTok, Discord, Telegram, YouTube, Reddit, WordPress, or Pinterest — or when managing campaigns, uploading media, or monitoring post delivery status.

x-api

X/Twitter API integration for posting tweets, threads, reading timelines, search, and analytics. Covers OAuth auth patterns, rate limits, and platform-native content posting. Use when the user wants to interact with X programmatically.

Git, Repo & Documentation

12 skills

architecture-decision-records

Capture architectural decisions made during Claude Code sessions as structured ADRs. Auto-detects decision moments, records context, alternatives considered, and rationale. Maintains an ADR log so future developers understand why the codebase is shaped the way it is.

code-tour

Create CodeTour `.tour`` files — persona-targeted, step-by-step walkthroughs with real file and line anchors. Use for onboarding tours, architecture walkthroughs, PR tours, RCA tours, and structured "explain how this works" requests. Use when the user asks for a code tour, onboarding walkthrough, PR tour, or an explanation of how a subsystem works.

codebase-onboarding

Analyze an unfamiliar codebase and generate a structured onboarding guide with architecture map, key entry points, conventions, and a starter CLAUDE.md. Use when joining a new project or setting up Claude Code for the first time in a repo.

codehealth-mcp

Real-time structural Code Health via CodeScene MCP — review before edits, verify score deltas after changes, gate commits and PRs. Use when reviewing code quality, refactoring, checking if AI changes degraded a file, or before commit/PR.

git-workflow

Git workflow patterns including branching strategies, commit conventions, merge vs rebase, conflict resolution, and collaborative development best practices for teams of all sizes. Use when choosing a branching strategy, writing commit conventions, deciding merge versus rebase, or resolving conflicts.

github-ops

GitHub repository operations, automation, and management. Issue triage, PR management, CI/CD operations, release management, and security monitoring using the gh CLI. Use when the user wants to manage GitHub issues, PRs, CI status, releases, contributors, stale items, or any GitHub operational task beyond simple git commands.

hookify-rules

This skill should be used when the user asks to create a hookify rule, write a hook rule, configure hookify, add a hookify rule, or needs guidance on hookify rule syntax and patterns.

inherit-legacy-style

Legacy-project style inheritance skill. Use when the user types `/inherit-legacy-style`, or when onboarding an AI coding agent onto a hand-written legacy project and you need to prevent "style drift" (the model imposing its pretrained mainstream idioms onto the project). Language- and framework-agnostic — it aligns meta-architecture only, not syntax. Once run, it becomes a behavioral constraint on all subsequent coding tasks. Do NOT use for pure research or one-off questions unrelated to code-style alignment.

living-docs-governance

Keep a long-lived project's documentation from rotting by assigning existing project docs clear constitution, map, status, and history roles, then wiring the active agent harness to those canonical sources. Use in the maintain

phase when docs drift from code, agents lose context between sessions, or intentional removals keep being recreated. Prefer adopting the repository's current docs structure over creating new root files.



plankton-code-quality

Write-time code quality enforcement using Plankton — auto-formatting, linting, and Claude-powered fixes on every file edit via hooks. Use when setting up write-time formatting, linting, or auto-fix hooks on file edits.

repo-scan

Bootstrap pointer that installs the external repo-scan skill from a pinned, reviewable commit. Use when repo-scan must be installed before running its cross-stack source-code asset audit; this ECC pointer does not perform the audit itself.

workspace-surface-audit

Audit the active repo, MCP servers, plugins, connectors, env surfaces, and harness setup, then recommend the highest-value ECC-native skills, hooks, agents, and operator workflows. Use when the user wants help setting up Claude Code or understanding what capabilities are actually available in their environment.

Integrations & Misc Tools

20 skills

api-connector-builder

Build a new API connector or provider by matching the target repo's existing integration pattern exactly. Use when adding one more integration without inventing a second architecture.

api-design

REST API design patterns including resource naming, status codes, pagination, filtering, error responses, versioning, and rate limiting for production APIs. Use when designing or reviewing REST endpoints, resource names, status codes, pagination, or versioning.

connections-optimizer

Reorganize the user's X and LinkedIn network with review-first pruning, add/follow recommendations, and channel-specific warm outreach drafted in the user's real voice. Use when the user wants to clean up following lists, grow toward current priorities, or rebalance a social graph around higher-signal relationships.

contract-first

Use when multiple consumers and providers must evolve an API or event schema without field drift, integration surprises, or one side silently redefining the interface.

dashboard-builder

Build monitoring dashboards that answer real operator questions for Grafana, SigNoz, and similar platforms. Use when turning metrics into a working dashboard instead of a vanity board.

documentation-lookup

Use up-to-date library and framework docs via Context7 MCP instead of training data. Activates for setup questions, API references, code examples, or when the user names a framework (e.g. React, Next.js, Prisma).

email-ops

Evidence-first mailbox triage, drafting, send verification, and sent-mail-safe follow-up workflow for ECC. Use when the user wants to organize email, draft or send through the real mail surface, or prove what landed in Sent.

exa-search

Neural search via Exa MCP for web, code, and company research. Use when the user needs web search, code examples, company intel, people lookup, or AI-powered deep research with Exa's neural search engine.

google-workspace-ops

Operate across Google Drive, Docs, Sheets, and Slides as one workflow surface for plans, trackers, decks, and shared documents. Use when the user needs to find, summarize, edit, migrate, or clean up Google Workspace assets without dropping to raw tool calls.

jira-integration

Use this skill when retrieving Jira tickets, analyzing requirements, updating ticket status, adding comments, or transitioning issues. Provides Jira API patterns via MCP or direct REST calls.

knowledge-ops

Knowledge base management, ingestion, sync, and retrieval across multiple storage layers (local files, MCP memory, vector stores, Git repos). Use when the user wants to save, organize, sync, deduplicate, or search across

their knowledge systems.

mailtrap-email-integration

Guides agents through integrating transactional email sending via Mailtrap's Email API, including sandbox testing, domain verification, and API authentication. Use when implementing email-sending features, debugging delivery issues, or setting up safe dev/staging email testing.

mcp-server-patterns

Build MCP servers with Node/TypeScript SDK — tools, resources, prompts, Zod validation, stdio vs Streamable HTTP. Use Context7 or official MCP docs for latest API. Use when building or debugging an MCP server — tools, resources, prompts, validation, or transport choice.

messages-ops

Evidence-first live messaging workflow for ECC. Use when the user wants to read texts or DMs, recover a recent one-time code, inspect a thread before replying, or prove which message source was actually checked.

nutrient-document-processing

Process, convert, OCR, extract, redact, sign, and fill documents using the Nutrient DWS API. Works with PDFs, DOCX, XLSX, PPTX, HTML, and images. Use when converting, OCRing, extracting from, redacting, signing, or filling documents via the Nutrient DWS API.

opensource-pipeline

Open-source pipeline: fork, sanitize, and package private projects for safe public release. Chains 3 agents (forker, sanitizer, packager). Triggers: 'opensource', 'open source this', 'make this public', 'prepare for open source'. Use when a private project must be forked, stripped of secrets, and packaged for public release.

product-capability

Translate PRD intent, roadmap asks, or product discussions into an implementation-ready capability plan that exposes constraints, invariants, interfaces, and unresolved decisions before multi-service work starts. Use when the user needs an ECC-native PRD-to-SRS lane instead of vague planning prose.

product-lens

Use this skill to validate the "why" before building, run product diagnostics, and pressure-test product direction before the request becomes an implementation contract.

project-flow-ops

Operate execution flow across GitHub and Linear by triaging issues and pull requests, linking active work, and keeping GitHub public-facing while Linear remains the internal execution layer. Use when the user wants backlog control, PR triage, or GitHub-to-Linear coordination.

unified-notifications-ops

Operate notifications as one ECC-native workflow across GitHub, Linear, desktop alerts, hooks, and connected communication surfaces. Use when the real problem is alert routing, deduplication, escalation, or inbox collapse.